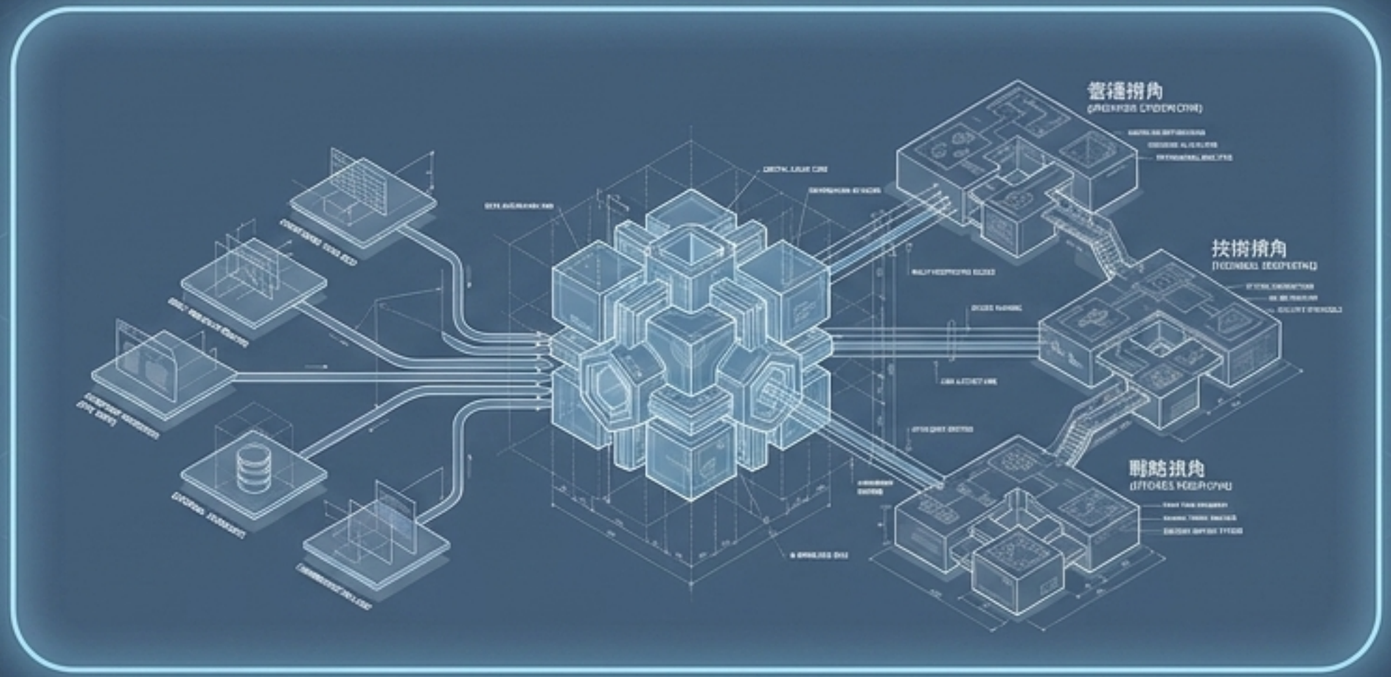


部門共用 AI 助理技術提案：構建數位神經中樞 (Digital Neural Hub)

以 Dify + LangGraph + MCP 混合架構，實現一次輸入、多路分發的營運中樞。

[Architectural Proposal & Technical Blueprint]

NotebookLM

現狀挑戰



相同長文本被多人各自的 Agent 重複讀取，導致 API 成本無意義飆升，且缺乏專案全局視角。

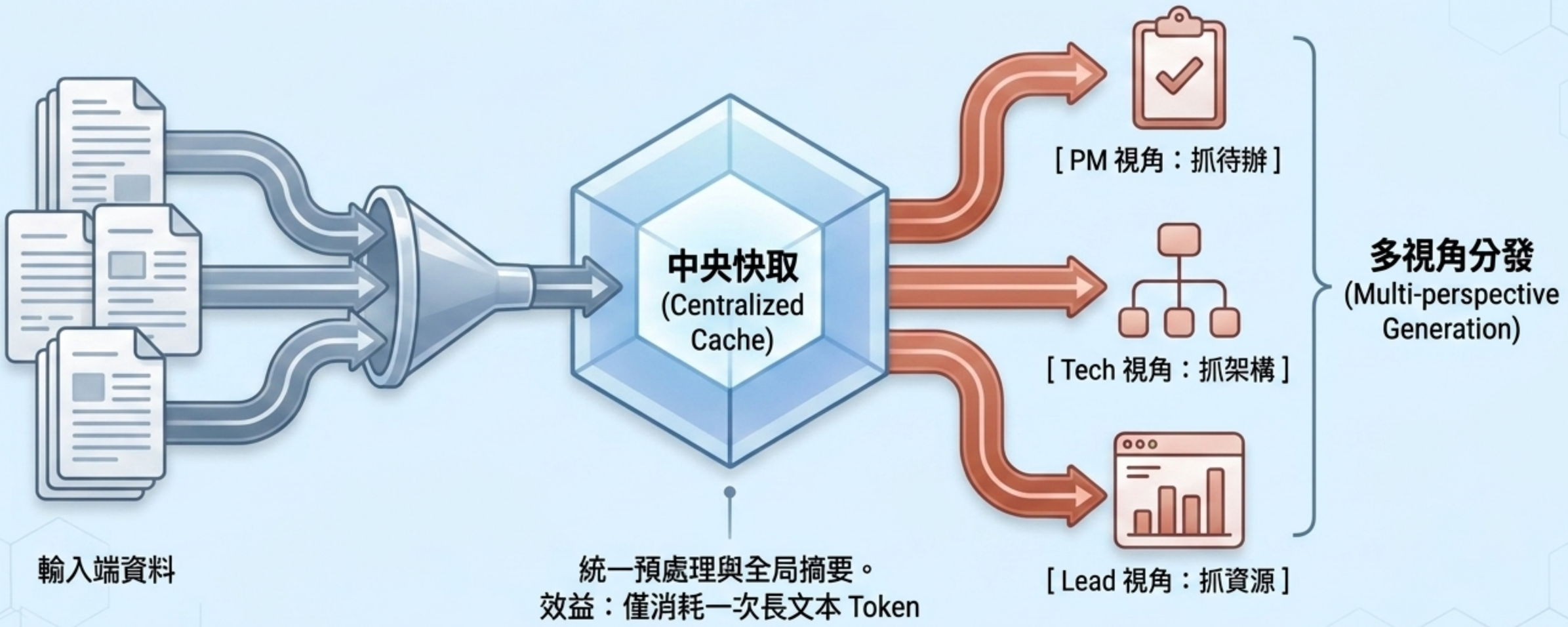
核心目標與終極解方



建立部門語境供應商。透過中央快取實現「一次輸入，多路輸出」，無縫對接終端。

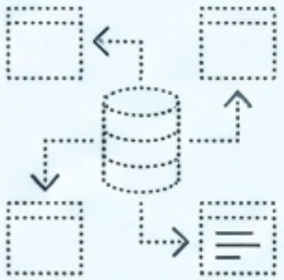
NotebookLM

架構願景：告別重複消耗，實現「一次輸入，多路分發」



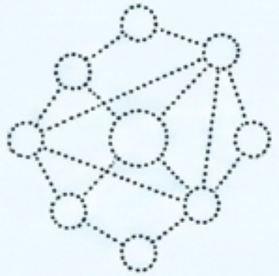
NotebookLM

數位神經中樞支援的四大核心業務場景



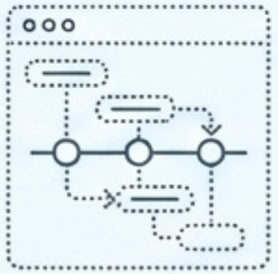
會議記錄統一轉譯與查詢

會議記錄統一 transcribe 一次，團隊成員可以個別依需求的資訊去 query。



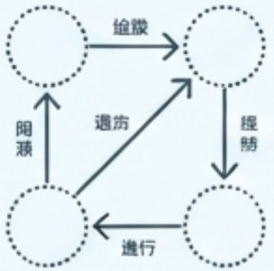
跨專案共用資源統一管理

部門共用的 inventory 統一管理：figma, design system, wiki，並做到 cross-project 資訊整合，減少不同人/不同專案需要去同一個檔案反覆爬取的浪費。



部門排程與籌備

比對多人空檔 (MSGraph)，透過多代理協作生成符合 SOP 的會議議程與籌備清單。

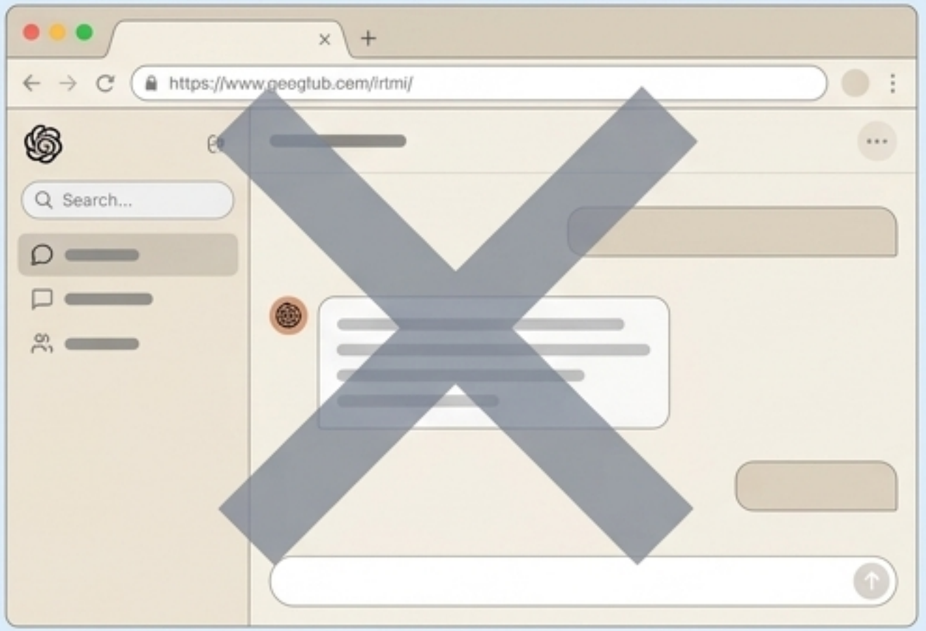


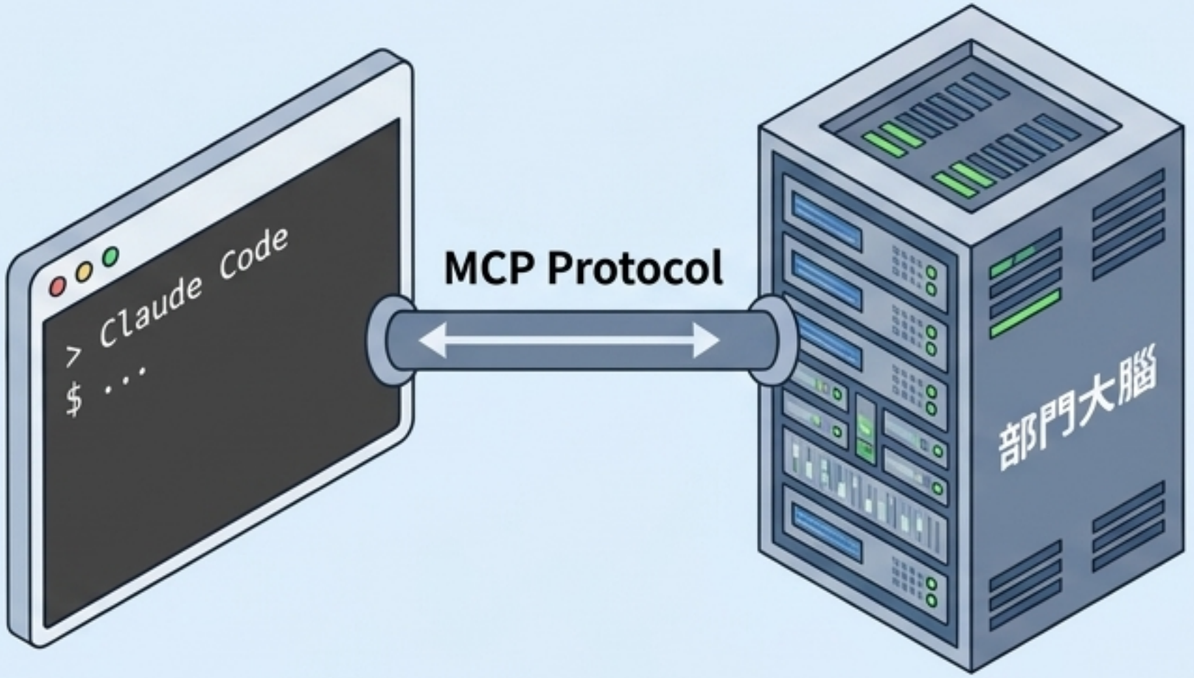
長期工作追蹤

運用 Checkpointers 追蹤任務狀態 (未開始/進行中/已完成)，支援狀態回溯與歷史記憶，確保任務不遺漏。

NotebookLM

互動典範轉移：從「對話視窗」走向「無縫超級工具」





[無縫終端體驗]

助理封裝為 MCP Server，直接成為開發者終端機內的超級工具，無需切換視窗。

[Agent-to-Agent 協作]

使用者在 Claude Code 的個人 Agent 透過 MCP 自動調用部門助理，獲取最新語境。

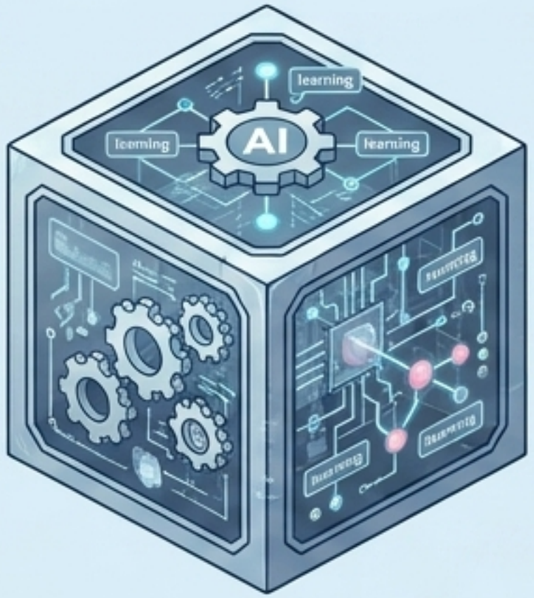
[消滅幻覺]

確保所有團隊成員的個人 Agent 都在一致的部門規範與即時進度下思考。

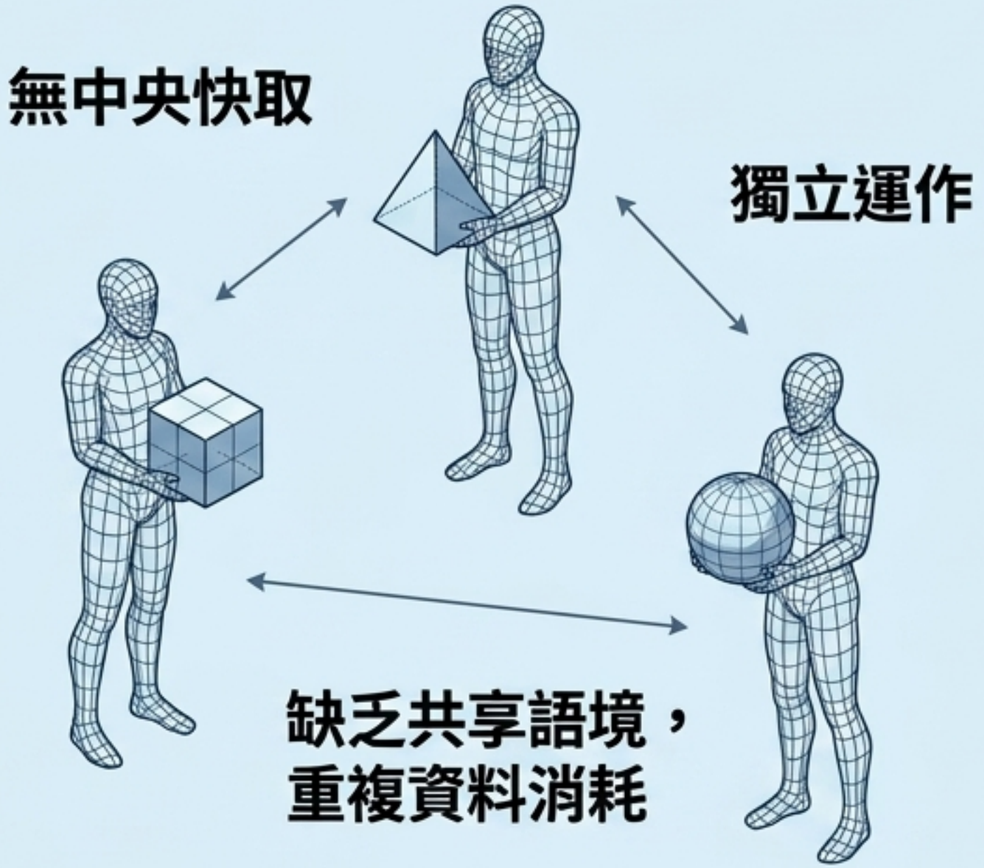
NotebookLM

痛點深挖：為何放棄「單體 Agent」框架？

單體 Agent 自進化



專注於個體自進化與單一 Agent 技能學習，但極易形成孤島



結論：本專案的核心痛點是『多人重複讀取資料』。我們需要的不是更聰明的單體 Agent，而是具備中央快取的『編排框架 (Workflow Engine)』。

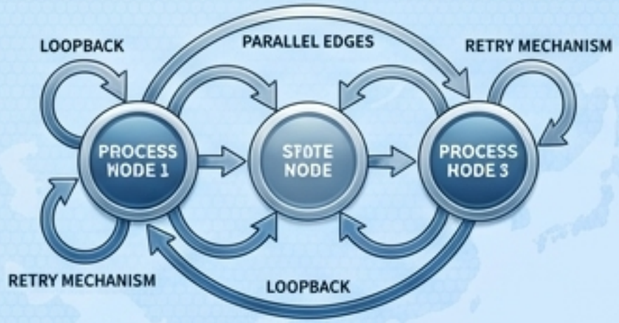
NotebookLM

技術選型矩陣：尋找最佳的「工作流引擎」與「數據基建」

CrewAI



LangGraph



維度	CrewAI	LangGraph	Dify
開發速度與交付	🟠 High	○ Low	🟠 High (拖拉介面)
狀態持久化與長效記憶	◑ Medium	🟠 High (Checkpointers)	◑ Medium
複雜邏輯與條件分支	◕ Medium	🟠 High (有環狀態機)	◑ Medium
RAG/知識庫管理門檻	○ Low	○ Low	🟠 High (內建切片向量化)
MCP 支援成熟度	○ Low	🟠 High	🟠 High

LangGraph 在「狀態持久化與複雜編排」具絕對優勢；Dify 在「低代碼 RAG 與數據工程」表現最佳。

NotebookLM

綜合建議：不要單選，要完美融合的立體架構



MCP（通訊接口層）

深度整合。將工作流封裝為 MCP Server，成為 Claude Code 的超級工具。

LangGraph（核心編排層）

精確控制。處理排程衝突、多路並發（Map-Reduce）與長效任務狀態追蹤。

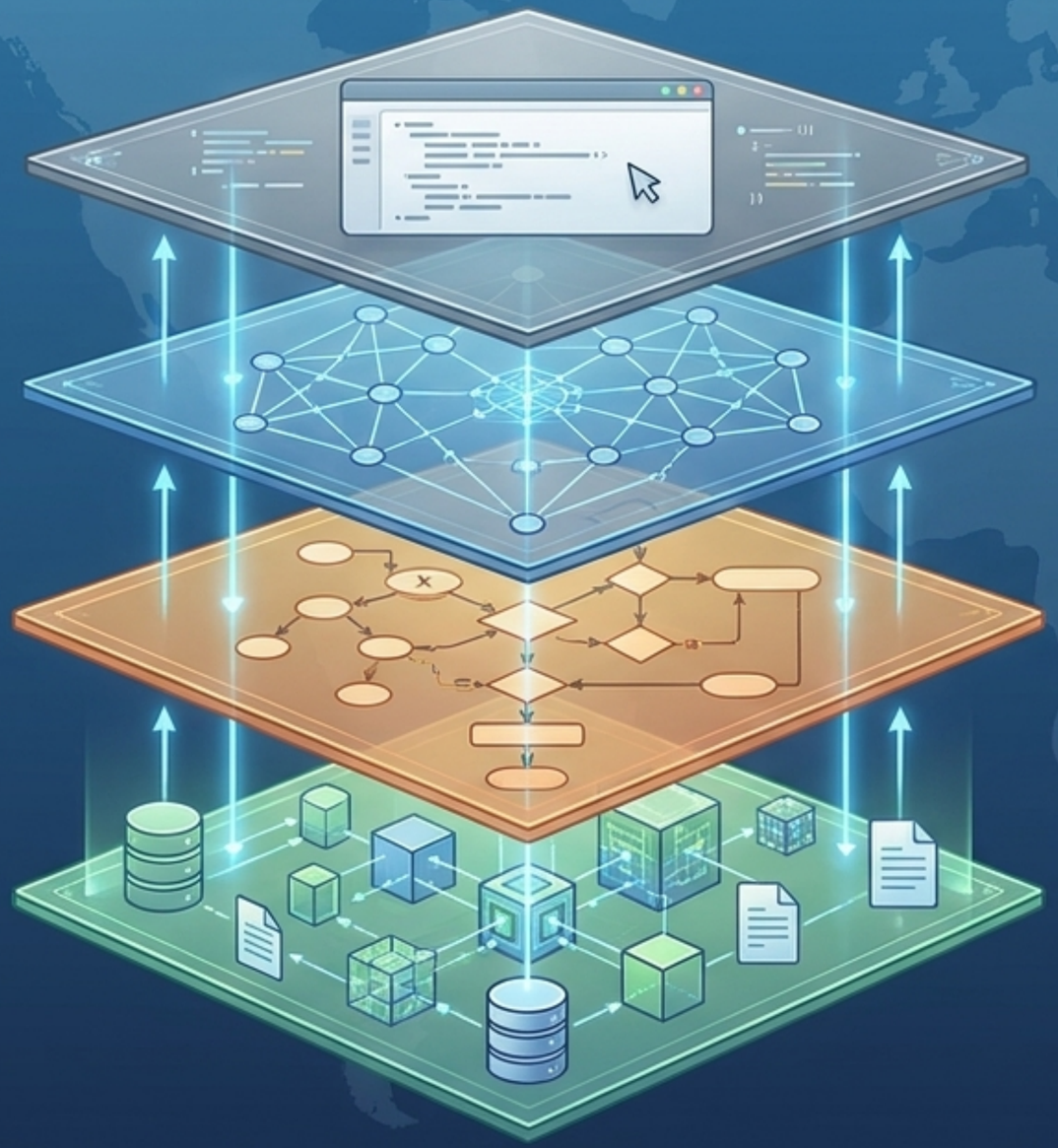
Dify（數據工程層）

敏捷交付。提供視覺化低代碼介面，輕鬆管理 RAG 知識庫與數據接入。

核心架構定義：以 Dify 為資料入口，LangGraph 為核心運算邏輯的 MCP 驅動架構。

NotebookLM

系統架構藍圖：四層式數位神經中樞



應用層 | Client (Claude Code)
使用者終端 (實現 Agent-to-Agent 自動調用)。

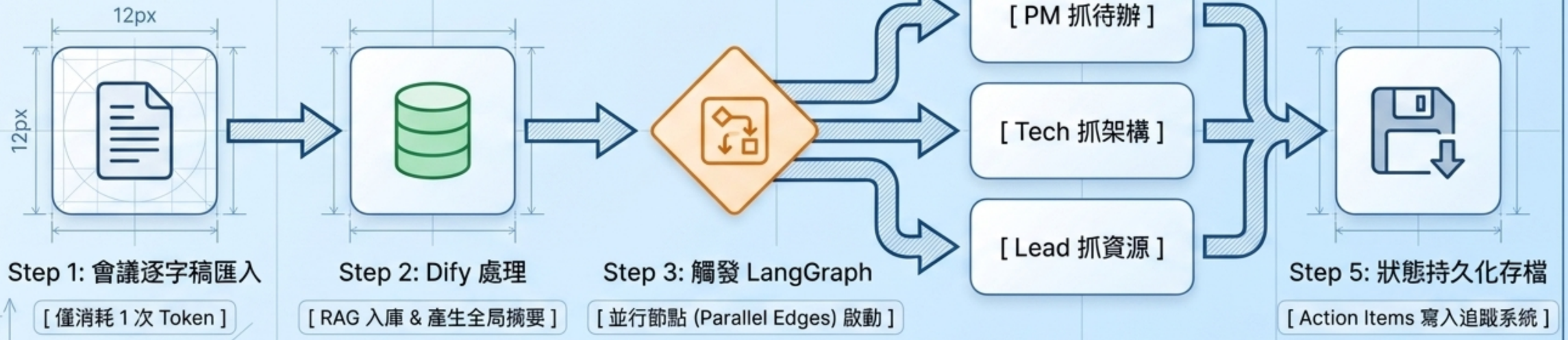
通訊層 | Protocol (MCP)
MCP Server (封裝 API，統一對外溝通口)。

中層 | Logic Layer (LangGraph)
任務狀態機，衝突排解，增量存儲 (Delta Channels)。

底層 | Data Layer (Dify)
會議記錄 , Confluence. , Figma 匯入 -> 自動切片 (Chunking) & 向量化。

NotebookLM

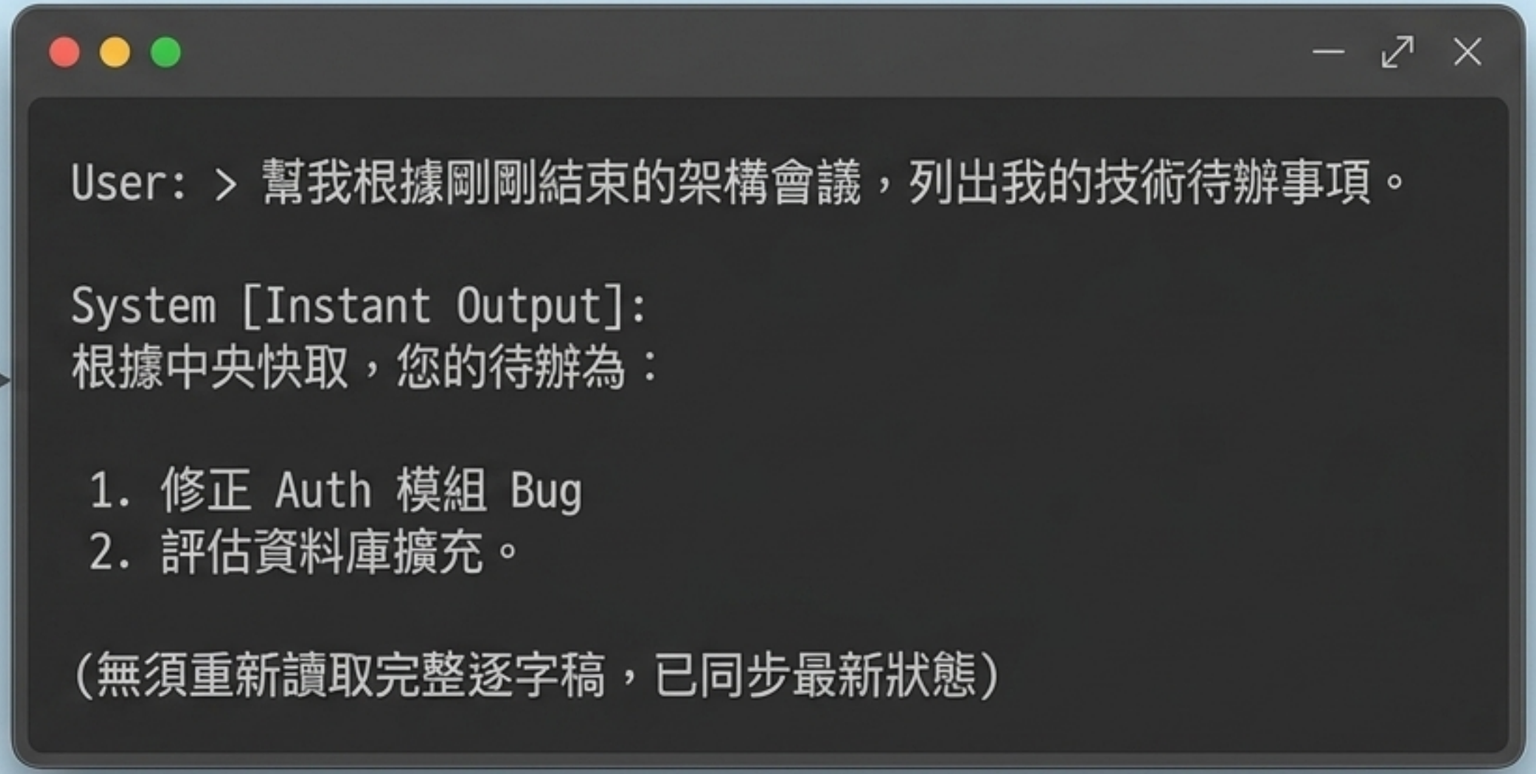
實戰演練：會議結束後的 5 步驟自動化工作流



NotebookLM

終端體驗：完全不需離開工作環境的 Agent-to-Agent 協作

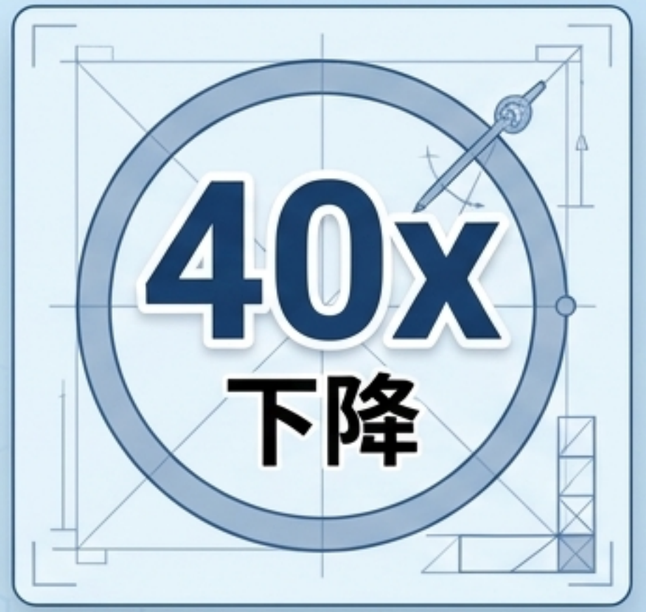
[個人 Agent 透過 MCP 向部門助理請求『Tech 視角』快取資料]



極致流暢。開發者不需開啟任何網頁或文件，資訊自動推送到指尖。

NotebookLM

商業價值與投資回報 (ROI)



成本極致優化

利用中央快取與增量存儲 (Delta Channels) 取代重複調用，API Token 成本呈指數級下降。



效率最大化

開發者不離開 Claude Code 即可掌握部門全局語境，徹底根絕跨平台 (Figma, Confluence) 的切換成本。



營運確定性

透過狀態持久化 (State Memory)，杜絕會議 Action Items 遺漏，實現長期任務狀態的完全掌握。

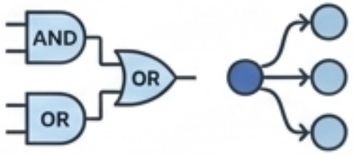
NotebookLM

實施路徑與下一步 (Roadmap & Next Steps)



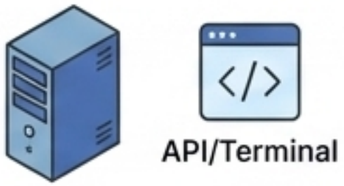
Phase 1: 數據基建 (Data Infra)

Dify RAG 知識庫建置與部門歷史數據初步接入。



Phase 2: 邏輯編排 (Logic Orchestration)

開發 LangGraph 核心工作流，定義 PM/Tech/Lead 多視角代理節點與狀態持久化機制。



Phase 3: 終端整合 (Client Integration)

MCP 伺服器封裝，於 Claude Code 進行內部端點測試與全體上線。

[專案準備就緒，隨時可啟動建立專屬「數位神經中樞」]

NotebookLM